BİL377 - HAFTA 05

Master Teoremi ve Böl-Yönet

Sıralama kodundan reküransa, rekürranstan çalışma maliyetine

Böl → Çöz → Birleştir

İki sıralı listeyi nasıl birleştiririz?

İki şubenin öğrenci numaraları kendi içinde sıralı. Tek sıralı liste gerekiyor.

Elimizdeki veri	Değerler
Sol liste	[2, 6, 7]
Sağ liste	[1, 3, 8]
Yan yana eklemek	[2, 6, 7, 1, 3, 8]
İstenen sonuç	[1, 2, 3, 6, 7, 8]

İlk adaylar 2 ve 1. En küçük eleman hangisi?

Birleştirme: her adımda ne değişti?

i sol listenin, j sağ listenin sıradaki elemanını gösterir. İndeksler 0'dan başlar.

Önce (i, j)	Karşılaştırma / karar	Karardan sonraki sonuç
(0, 0)	2 ≤ 1 yanlış → sağdan 1	[1]
(0, 1)	2 ≤ 3 doğru → soldan 2	[1, 2]
(1, 1)	6 ≤ 3 yanlış → sağdan 3	[1, 2, 3]
(1, 2)	6 ≤ 8 doğru → soldan 6	[1, 2, 3, 6]
(2, 2)	7 ≤ 8 doğru → soldan 7	[1, 2, 3, 6, 7]
(3, 2)	Sol bitti → kalan 8'i ekle	[1, 2, 3, 6, 7, 8]

5 eleman karşılaştırması; sonuca toplam 6 eleman yazıldı.

Birleştirmenin çalışan Python kodu


```
def birlestir(sol, sag):
    i = j = 0
    sonuc = []
    while i < len(sol) and j < len(sag):
        if sol[i] <= sag[j]:
            sonuc.append(sol[i])
            i += 1
        else:
            sonuc.append(sag[j])
            j += 1
    sonuc.extend(sol[i:])
    sonuc.extend(sag[j:])
    return sonuc
```

kod/ders_ornekleri.py → birlestir

Döngünün görevi

İki tarafta da eleman varken küçük olanı seç.

Bir taraf bitince diğerinin kalanını ekle.

Giriş listeleri değişmez.

En küçük adayı seçmek neden doğru?

Döngü değişmezi: sonuç, alınmış elemanların sıralı dizisidir; kalanlardan büyük değildir.

Başlangıç: sonuç boş; hiçbir eleman atlanmadı.

Adım: kalanların en küçüğü sol[i] veya sag[j].

Seçim: küçüğünü ekle; yalnızca o indeksi ilerlet.

Bitiş: kalan sıralı parça güvenle sona eklenir.

Her eleman bir kez alınır: eleman kaybı ve fazladan kopya yoktur.

Birleştirme neden Θ(n)?

n = len(sol) + len(sag). Burada n, iki listenin toplam eleman sayısıdır.

Saydığımız şey	Bu kodda olan
İndekslerin hareketi	Her turda i veya j bir artar.
Eleman karşılaştırması	İki taraf da doluyken yapılır; en fazla n – 1.
Çıktıya yazma	Kalanlar dahil tam n eleman yazılır.
Birleştirmenin toplam işi	Alt sınır Ω(n), üst sınır O(n): Θ(n).

Örnekte n = 3 + 3 = 6. 5 karşılaştırma olsa da 6 eleman üretilir.

Sıralı olmayan listeyi küçültelim

Girdi: [7, 2, 6, 3, 8, 1, 5, 4]. Tek elemanlı liste zaten sıralıdır.

Derinlik	Parçalar	Parça boyutu
0	[7,2,6,3,8,1,5,4]	8
1	[7,2,6,3] [8,1,5,4]	4
2	[7,2] [6,3] [8,1] [5,4]	2
3	[7] [2] [6] [3] [8] [1] [5] [4]	1

Bölme sıralamaz. Sıralı sonuçlar geri dönüşte birleşir.

Geri dönüşte sıralama oluşur

Her birleşmede iki giriş de sıralıdır. Birleştirme bunu bir üst boyuta taşır.

Birleşen boyut	Elde edilen sıralı parçalar	Yazılan eleman
1 + 1 → 2	[2,7] [3,6] [1,8] [4,5]	4 × 2 = 8
2 + 2 → 4	[2,3,6,7] [1,4,5,8]	2 × 4 = 8
4 + 4 → 8	[1,2,3,4,5,6,7,8]	1 × 8 = 8

8 + 8 + 8 = 24 birleştirme yazması.

Merge Sort: böl, çöz, birleştir


```
def merge_sort(dizi):
    if len(dizi) <= 1:
        return dizi.copy()
    orta = len(dizi) // 2
    sol = merge_sort(dizi[:orta])
    sag = merge_sort(dizi[orta:])
    return birlestir(sol, sag)
```

Üç ayrı iş

Böl: orta ve iki dilim

Çöz: iki özyinelemeli çağrı

Birleştir: sıralı sonuçları bir araya getir

Taban: 0 veya 1 eleman. Girdi korunur, sıralı kopya döner.

Kod hangi sırayla geri döner?

Küçük izleme: merge_sort([7, 2, 6, 3]). Her çağrı bitince kaldığı satıra döner.

Sıra	Tamamlanan işlem	Üst çağrıya dönen değer
1	merge_sort([7]) ve merge_sort([2])	[7] ve [2]
2	birlestir([7], [2])	[2, 7]
3	merge_sort([6]) ve merge_sort([3])	[6] ve [3]
4	birlestir([6], [3])	[3, 6]
5	birlestir([2, 7], [3, 6])	[2, 3, 6, 7]

7 çağrı vardır: 1 kök + 2 ara çağrı + 4 taban çağrısı.

Sekiz elemanda ölçülen değerler

Girdi: [7, 2, 6, 3, 8, 1, 5, 4]. Sayaçlar çalışan koddan alındı.

Ölçüm	Değer	Nereden geliyor?
Eleman karşılaştırması	17	Alt birleşmeler: 4 + 6; kök: 7
Birleştirme yazması	24	8 + 8 + 8
merge_sort çağrısı	15	1 + 2 + 4 + 8
En büyük derinlik	3	8 → 4 → 2 → 1

17, 24, 15 ve 3 farklı soruların cevaplarıdır.

Aynı n, farklı karşılaştırma sayısı

Bölünme biçimi değişmiyor. Birleşmede bir tarafın ne zaman tükendiği değişiyor.

Girdi	Karşılaştırma	Yazma	Çağrı
[1,2,3,4,5,6,7,8]	12	24	15
[8,7,6,5,4,3,2,1]	12	24	15
[7,2,6,3,8,1,5,4]	17	24	15

Bu Merge Sort, sıralı girdide de Θ(n log n) çalışır.

Koddan reküransı çıkaralım

n çiftken iki parça n/2 elemanlıdır. T(n), toplam çalışma maliyetini temsil eder.

Kodun bölümü	Maliyet	Gerekçe
Sol parçayı sırala	T(n/2)	Yarım boyutlu bir problem
Sağ parçayı sırala	T(n/2)	İkinci yarım problem
Dilimleri oluştur ve birleştir	Θ(n)	Toplam n eleman üzerinden iş
Toplam	2T(n/2) + Θ(n)	İki alt çağrı ve çağrı dışı iş

Taban: T(0), T(1) = Θ(1). n tekse parçalar ⌊n/2⌋ ve ⌈n/2⌉.

T(8) = 32 nereden geliyor?

Basitleştirilmiş model: T(n)=2T(n/2)+n, T(1)=1. Bu model tüm Python işlemlerini saymaz.

$$T(2) = 2 \cdot T(1) + 2 = 2 \cdot 1 + 2 = 4$$

$$T(4) = 2 \cdot T(2) + 4 = 2 \cdot 4 + 4 = 12$$

$$T(8) = 2 \cdot T(4) + 8 = 2 \cdot 12 + 8 = 32$$

32 = 24 birleştirme işi + 8 taban birimi

Bu modelde $T(n)=n \log_2 n+n$. Kodda 17 karşılaştırma bulunmasıyla çelişmez.

n log n çarpanı nereden çıktı?

n = 2^h olsun. h, boyutun 1'e inmesi için gereken yarılama sayısıdır.

Derinlik i	Parça sayısı	Parça başına iş	Düzey toplamı
0	1	n	n
1	2	n/2	n
2	4	n/4	n
i	2^i	n / 2^i	n
$h = \log_2 n$	n taban	1	n

h iç düzey × n iş + n taban = $n \log_2 n + n \rightarrow \Theta(n \log n)$.

Tek boyutlar ve bellek maliyeti

Gerçek Python kodunda dilim kopyalama da vardır. Sayaç ile bellek aynı şey değildir.

Durum / kaynak	Ne olur?
n=7	İlk bölme 3 + 4; tabana yine ulaşılır.
Dilimleme ve birleşme	Her iç çağrıda toplam Θ(n) yerel iş.
Aynı anda gereken liste alanı	Θ(n) ek alan; bütün geçmiş listeler tutulmaz.
Çağrı yığını	Θ(log n) derinlik; toplam çağrı sayısı Θ(n).

Bu uygulama yerinde sıralama yapmaz. Zaman: Θ(n log n).

O, Ω ve Θ: bu derste nasıl okunur?

Örnek maliyet: $g(n)=3n+2$. $n \geq 1$ için $3n \leq 3n+2 \leq 5n$.

Gösterim	Okunuş / anlam	Örnekte gerekçe
$g(n) = O(n)$	Büyük O: doğrusal üst sınır	$3n+2 \leq 5n$
$g(n) = \Omega(n)$	Omega: doğrusal alt sınır	$3n \leq 3n+2$
$g(n) = \Theta(n)$	Theta: doğrusal sıkı sınır	İki sınır birlikte geçerli

O “en kötü”, Ω “en iyi” demek değildir; bir fonksiyona sınır koyarlar.

a, b ve f(n) tam olarak ne?

Genel biçim: T(n) = aT(n/b) + f(n). Bir çağrının toplam işini parçalıyoruz.

Sembol	Sorduğumuz soru	Merge Sort
a	Gerçekte kaç alt çağrı çalışıyor?	2
b	Her alt boyut n'in kaçta biri?	n/2 → b=2
f(n)	Alt çağrıların dışında ne iş var?	Dilimleme + birleştirme: Θ(n)
T(1)	En küçük problem neye mal oluyor?	Θ(1)

a ve b sabit olmalı; a≥1, b>1. f(n) sonunda pozitif olmalı.

log_b a nasıl hesaplanır?

p = log_b a demek, b^p = a demektir: “b’nin hangi kuvveti a eder?”

a	b	Aradığımız eşitlik	p = log_b a
2	2	$2^1 = 2$	1
4	2	$2^2 = 4$	2
16	4	$4^2 = 16$	2
1	2	$2^0 = 1$	0
3	2	2^p = 3	$\log_2 3 \approx 1{,}585$

p bir üs, n^p bir fonksiyon. Örneğin p=2 ise karşılaştırma ölçeği n^2.

Neden n^(log_b a) ile karşılaştırıyoruz?

n=b^h kabul edelim. Her düğüm a çocuk üretir; her alt boyut b'ye bölünür.

Tabanda: n / b^h = 1 → h = log_b n

Yaprak sayısı: a^h = a^(log_b n)

p = log_b a ise a = b^p; a^h = (b^h)^p = n^p

Sabit taban maliyeti → toplam yaprak işi Θ(n^p)

Örnek: a=4, b=2, n=8 → h=3 → $4^3=64=8^2$ yaprak.

Aynı a ve b; üç farklı büyüme

$T(n)=2T(n/2)+f(n)$, $T(1)=1$, $n=8$. Her hücre bir düzeyin toplam işidir.

Derinlik	$f(n)=1$	$f(n)=n$	$f(n)=n^2$
0: tek düğüm	1	8	64
1: iki düğüm	2	8	32
2: dört düğüm	4	8	16
3: sekiz taban	8	8	8
Toplam	15	32	120

İş aşağı doğru artıyor, aynı kalıyor veya azalıyor. Üç durumun kaynağı bu.

Önce kolay durum: f(n)=Θ(n^d)

p=log_b a. Yerel işin üssü d ile yaprak işinin üssü p'yi karşılaştır.

Üsler	Ağaçta olan	Sonuç
d < p	Yaprak tarafı baskın	Θ(n^p)
d = p	Düzeylerin işi aynı mertebede	Θ(n^p log n)
d > p	Köke yakın iş baskın	Θ(n^d)

Bu kısa tablo yalnızca saf kuvvet biçimi için: f(n)=Θ(n^d).

Durum 1: yaprak işi baskın

$T(n)=4T(n/2)+n$. Her düğüm dört yarım boyutlu alt problem çözüyor.

$a=4$, $b=2$, $f(n)=n$

$p=\log_2 4=2 \rightarrow n^p=n^2$

$n = n^{(2-1)}$: pozitif üs farkı $\varepsilon=1$

$T(n)=\Theta(n^2)$

f(n) tek başına doğrusal; ama çağrı ağacının toplam işi doğrusal değil.

Durum 1’i sayılarla görelim

T(n)=4T(n/2)+n, T(1)=1. n=16 için boyut dört kez yarılanır.

Derinlik	Düğüm sayısı	Tek düğüm işi	Düzey toplamı
0	1	16	16
1	4	8	32
2	16	4	64
3	64	2	128
4: taban	256	1	256

16+32+64+128+256 = 496. Genel model: $2n^2-n \rightarrow \Theta(n^2)$.

Durum 2: düzeyler dengeli

T(n)=2T(n/2)+n. Merge Sort için kullandığımız model.

$a=2$, $b=2$, $p=\log_2 2=1$

f(n)=n=Θ(n^p)

Her iç düzey Θ(n); iç düzey sayısı $\log_2 n$

T(n)=Θ(n log n)

Eşitlik durumunda bir log çarpanı eklenir; nedeni düzeylerin toplanmasıdır.

Durum 3: kökteki iş baskın

$T(n)=2T(n/2)+n^2$. Bölünme aynı, çağrı dışındaki iş artık karesel.

$a=2, b=2, p=1;\ f(n)=n^2$

$n^2 = n^{(1+1)}$: pozitif üs farkı $\varepsilon=1$

$2f(n/2)=2(n/2)^2=n^2/2=(1/2)f(n)$

$T(n)=\Theta(n^2)$

Düzenlilik sağlandı: $c=1/2<1$. Üs farkı ve düzenlilik birlikte kontrol edilir.

Düzenlilik şartı ne söylüyor?

a f(n/b) ≤ c f(n), sabit 0<c<1. Burada alt çağrıların yalnızca yerel işleri karşılaştırılır.

f(n)=n^2 ve a=b=2 olsun.

n=8: kökün yerel işi 64.

İki çocuğun yerel işleri: 16+16=32.

32 ≤ (1/2)·64: aşağıdaki düzey küçülüyor.

c, koddan gelen bir sayaç değildir; eşitsizliği sağlayan sabit çarpandır.

ε: “biraz daha büyük” yeterli mi?

ε (epsilon), n’den bağımsız pozitif bir üs farkıdır. Değeri 1 olmak zorunda değildir.

Karşılaştırma	Oran	Polinom farkı var mı?
n^2 ile n	n	Evet: üs farkı 1
n^0,51 ile n^0,5	n^0,01	Evet: üs farkı 0,01
n log n ile n	log n	Hayır: sabit pozitif üs farkı yok
n ile n/log n	log n	Hayır: sabit pozitif üs farkı yok

“Daha hızlı büyüyor” ile “polinom derecesinde daha hızlı büyüyor” aynı koşul değildir.

Master Teoremi: temel üç durum

$T(n)=aT(n/b)+f(n)$, $p=\log_b a$; $a\geq 1$ ve $b>1$ sabit, taban $\Theta(1)$.

1. $f(n)=O(n^{(p-\varepsilon)})$, $\varepsilon>0 \rightarrow T(n)=\Theta(n^p)$
2. $f(n)=\Theta(n^p) \rightarrow T(n)=\Theta(n^p \log n)$
3. $f(n)=\Omega(n^{(p+\varepsilon)})$, $\varepsilon>0$ ve düzenlilik

   $a\, f(n/b) \leq c\, f(n)$, $c<1 \rightarrow T(n)=\Theta(f(n))$

Önce biçim, sonra p, sonra koşul; en son sonuç.

Çözümlü örnek: $3T(n/2)+n^2$

Yerel Master Teoremi soru seti, soru 1. Her koşulu ayrı kontrol edelim.

$a=3$, $b=2$, $p=\log_2 3 \approx 1{,}585$; $f(n)=n^2$

$2 > p$; $\varepsilon = 2-\log_2 3 \approx 0{,}415 > 0$

$3f(n/2)=3(n/2)^2=(3/4)f(n)$; $c=3/4<1$

Durum 3 → $T(n)=\Theta(n^2)$

3 alt çağrı olması tek başına n^3 sonucu vermez.

Çözümlü örnek: 16T(n/4)+n

Yerel soru seti, soru 5. a ile b'yi birbirinden bağımsız okuyalım.

$a=16$, $b=4$, $f(n)=n$

$4^2=16 \rightarrow p=\log_4 16=2 \rightarrow n^p=n^2$

$f(n)=n=O(n^{(2-1)})$; $\varepsilon=1$

Durum 1 → $T(n)=\Theta(n^2)$

n=16: boyut 16→4→1, düğüm sayısı 1→16→256.

Çözümlü örnek: 3T(n/2)+n

Yerel soru seti, soru 12. Sonucun üssü tam sayı olmak zorunda değil.

$a=3,\ b=2,\ p=\log_2 3 \approx 1{,}585$

$f(n)=n;\ \ 1 < \log_2 3$

$\varepsilon=\log_2 3-1>0 \rightarrow n=O(n^{(p-\varepsilon)})$

Durum 1 → $T(n)=\Theta(n^{(\log_2 3)})$

n=8: yaprak sayısı $3^3=27$. Çünkü $8^{(\log_2 3)}=27$.

Tek alt çağrı varsa: a=1

İkili aramanın en kötü yol modeli: T(n)=T(n/2)+1, T(1)=1.

a=1, b=2; $p=\log_2 1=0$

$n^p=n^0=1$; $f(n)=1=\Theta(n^0)$

Durum 2: $\Theta(n^0 \log n)=\Theta(\log n)$

n=8: T(8)=T(4)+1=T(2)+2=T(1)+3=4

İki seçenek bulunması iki çağrı yapılması demek değildir.

Önce yalnız a, b ve f(n)'i bulun

A. $T(n) = 4T(n/2) + n$

B. $T(n) = 2T(n/4) + n$

C. $T(n) = T(n/2) + n$

D. $T(n) = 2T(n/2) + n^2$

Her biri için p=log_b a değerini de hesaplayın. Henüz sonucu ezberden yazmayın.

Aynı görünen reküranslar, farklı sonuçlar

Önceki dört sorunun kontrolü. Üçüncü durumda düzenlilik oranını da yazıyoruz.

Soru	p	Durum / gerekçe	Sonuç
A: $4T(n/2)+n$	2	1: n, n^2'den küçük	$\Theta(n^2)$
B: $2T(n/4)+n$	1/2	3: $2f(n/4)/f(n)=1/2$	$\Theta(n)$
C: $T(n/2)+n$	0	3: $f(n/2)/f(n)=1/2$	$\Theta(n)$
D: $2T(n/2)+n^2$	1	3: $2f(n/2)/f(n)=1/2$	$\Theta(n^2)$

"İkiye bölüyor" bilgisi tek başına çalışma sınıfını belirlemez.

İkinci durumun logaritmalı biçimi

Yerel kaynakta kullanılan genişletilmiş ikinci durum: sabit k≥0.

$$f(n)=\Theta(n^p \cdot (\log n)^k)$$

Sonuç: $T(n)=\Theta(n^p \cdot (\log n)^{(k+1)})$

Örnek: $2T(n/2)+n \log n \rightarrow p=1, k=1$

$T(n)=\Theta(n (\log n)^2)$

$\log^2 n$, $(\log n)^2$ demektir; $\log(n^2)$ değildir.

n log n yerel işten log² nasıl doğar?

$T(n)=2T(n/2)+n\log_2 n$, $T(1)=1$. Örnek $n=8$, $h=\log_2 8=3$.

Derinlik	Düğüm başına iş	Düğüm sayısı	Düzey toplamı
0	$8\cdot\log_2 8 = 24$	1	24
1	$4\cdot\log_2 4 = 8$	2	16
2	$2\cdot\log_2 2 = 2$	4	8
3: taban	1	8	8

İç iş: $8(3+2+1)=48$. Tabanlarla 56; genelde $\Theta(n\,(\log n)^2)$.

Master her reküransa uygulanmaz

Önce $T(n)=aT(n/b)+f(n)$ biçimini ve sabitleri kontrol et.

Rekürans	Bu sürüm neden uygulanamaz?
$T(n)=T(n-1)+n$	Boyut sabit oranda değil, 1 azalıyor.
$T(n)=T(n/3)+T(2n/3)+n$	Alt problem boyutları eşit değil.
$T(n)=nT(n/2)+n$	$a=n$ sabit değil.
$T(n)=2T(n/2)-n$	Yerel iş $f(n)=-n$ pozitif değil.

“Master uygulanamaz” demek “rekürans çözülemez” demek değildir.

Sınır örneği: 2T(n/2)+n/log n

a=2, b=2, p=1. f(n), n'den küçük ama polinom derecesinde küçük değil.

Durum 1? (n/log n) / n^(1–ε) = n^ε/log n

Her sabit ε>0 için oran sınırsız büyür → koşul yok.

Durum 2? k=–1; kullandığımız k≥0 sürümüne uymaz.

Durum 3? n'den büyük değil → koşul yok.

Bu Master sürümü uygulanamaz. Ağaç toplamı ayrıca çözülebilir.

Koddan analiz: toplamı iki parçaya bölmek


```
def toplam(dizi, bas, son):
    if son == bas:
        return 0
    if son - bas == 1:
        return dizi[bas]
    orta = (bas + son) // 2
    sol = toplam(dizi, bas, orta)
    sag = toplam(dizi, orta, son)
    return sol + sag
```

Aralık: [bas, son)

son konumu dahil değil.

Dilim kopyası yok; indeksleri geçiriyoruz.

a, b ve f(n) nedir?

Sonuç neden n log n değil?

Çağrı: toplam([7, 2, 6, 3], 0, 4) → 18

Aynı bölme, farklı çağrı dışı iş

İki yöntemde de a=2 ve b=2. Sonucu değiştiren f(n).

Algoritma	Çağrı dışı iş	Rekürans	Sonuç
İndeksli toplam	İki sayıyı toplamak	$2T(n/2)+\Theta(1)$	$\Theta(n)$
Merge Sort	Dilimleme + birleşme	$2T(n/2)+\Theta(n)$	$\Theta(n \log n)$

İndeksli toplamı dilimlerle yeniden yazarsak kopyalama maliyeti eklenir.

Derste kodu çalıştırıp sayalım


```
from ders_ornekleri import olcumlu_sirala

dizi = [7, 2, 6, 3, 8, 1, 5, 4]
sonuc, sayac = olcumlu_sirala(dizi)
print(sonuc)
print(sayac.karsilastirma)  # 17
print(sayac.yazma)          # 24
print(sayac.cagri)          # 15
print(sayac.en_buyuk_derinlik)  # 3
```

Girdiyi değiştir

Sıralı 8 eleman
→ 12 karşılaştırma
→ 24 yazma

Sıralı 16 eleman
→ 32 karşılaştırma
→ 64 yazma

kod klasöründe: python3 ders_ornekleri.py olcum

Koddaki değişiklik hangi hatayı üretir?

1. İki extend satırını da silersek ne kaybolur?

2. Seçimden sonra i ve j'yi birlikte artırırsak?

3. <= yerine < yazarsak sıralama bozulur mu?

4. Taban koşulunu yalnız len(dizi)==1 yaparsak?

Her cevap için küçük bir girdiyle davranışı gösterin.

Dört reküransı gerekçesiyle çözümü

A. $T(n) = 9T(n/3) + n^2$

B. $T(n) = 4T(n/2) + n^3$

C. $T(n) = T(n/2) + 1$

D. $T(n) = 2T(n-1) + 1$

a, b, f(n), p ve koşul yazılır. Uymuyorsa nedenini belirtilir.

Çözüm kontrolü

Sonuçtan önce hangi koşulun sağlandığını okuyun.

Soru	Belirleyici adım	Sonuç
$9T(n/3)+n^2$	$p=2$, $f(n)=\Theta(n^2)$: durum 2	$\Theta(n^2 \log n)$
$4T(n/2)+n^3$	$p=2$, $\varepsilon=1$, düzenlilik oranı 1/2	$\Theta(n^3)$
$T(n/2)+1$	$p=0$, $f(n)=\Theta(1)$: durum 2	$\Theta(\log n)$
$2T(n-1)+1$	Sabit oranlı küçülme yok	Master uygulanmaz

Son satır açma yöntemiyle Θ(2^n) verir; yöntem seçimi de çözümün parçasıdır.

Bir rekürans çözümünün kontrol listesi

1. Koddan a, b ve f(n)'i doğru çıkardım mı?

2. p=log_b a hesabını b^p=a ile doğruladım mı?

3. Üs farkını / eşitliği ve gerekirse düzenliliği gösterdim mi?

4. Tam sayaçla Θ büyüme sınıfını birbirinden ayırdım mı?

Merge Sort: a=2, b=2, f(n)=Θ(n) → Θ(n log n).

Ödev: listedeki ters çiftleri sayma

Ters çift: i<j olduğu halde dizi[i]>dizi[j]. Eşit değerler ters çift değildir.

Örnek	Ters çiftler / beklenen sayı
[3, 1, 2]	(3,1), (3,2) → 2
[1, 2, 3]	Ters çift yok → 0
[3, 2, 1]	(3,2), (3,1), (2,1) → 3
[2, 2, 1]	İki farklı 2 konumu, 1 ile çift → 2

Görev: Merge Sort'u uyarlayın; $\Theta(n^2)$ çift taramasıyla doğruluğunu karşılaştırın.